

SW Engineering CSC648-848 Spring 2026

Gator Freighter

Team 14

Name	Role
Leslie Garcia, lgarcia26@sfsu.edu	Team Lead
Jordan Westover	Front-End Lead
Stiofan Condon	Back-End Lead
Gursimran Grewal	GitHub Maintainer
Kiran Khatri	Full-stack contributor
Gabriel Purizaca	Full-stack contributor
Christopher Huynh	

Milestone 4

Date	May 18, 2026
Date Submitted	May 22, 2026

Table Of Contents

1. <u>Product Summary</u>	3
2. <u>Usability test plan for selected function</u>	4
3. <u>QA test plan and QA testing</u>	6
4. <u>Peer Code Review</u>	9
5. <u>Self-check on best practices for security</u>	32
6. <u>Self-check of the adherence to original Non-function specs</u>	33

1. Product Summary:

Welcome to Gator Freightier! A student-first marketplace built exclusively for San Francisco State University students. Whether you're looking to sell last semester's textbooks, pick up dorm essentials, or trade gear with a fellow Gator, our platform makes it simple, safe, and local. Unlike general marketplaces where anything goes, Gator Freightier is a close community – every user is verified as a current SFSU student or faculty member, so you can always know exactly who you're dealing with. Transactions stay on campus too, with designated meet-up spots that make exchanges convenient and secure. No strangers, no sketchy meetups, no hassle. We also offer built-in messaging so buyers and sellers can connect directly without leaving the app. Gator Freightier is a smarter, safer way for the SFSU community to support one another, cut costs, and keep resources circulating right where they belong: on campus.

Itemized list of major committed functions: (1 line each)

- Students can browse and search through listings using keywords, categories, or specific courses to find what they need.
- Anyone looking to sell or trade can create a listing in just a few steps, complete with photos, a description, and a price.
- Buyers can reach out directly to sellers through built-in messaging
- Students sign up using their SFSU email, so anybody using the app stays verified.
- When a deal is ready, both parties choose a designated meet-up spot.
- Sellers can manage, edit or delete their listings at any time from their dashboard.
- Students can send trade requests to exchange items directly with one another.
- Each student has their own account dashboard to track their listings, messages, and all activity.

Link to our product: <https://team14.csc648sfsu.com/>

2. Usability test plan for selected function

Selected function: Create a listing

Usability plan:

1. Test objectives:

The primary objective is to evaluate how effectively and efficiently SFSU students and faculty can create a new listing on Gator Freight. Specifically, the test aims to

- Determine whether users can successfully locate and complete the Create a Listing feature without external guidance.
- Identify any usability barriers, confusing UI elements, or unclear instructions that impede the listing creation process.
- Measure the time and number of steps required for a new user to publish a listing from start to finish.
- Assess user satisfaction and confidence after completing the listing creation task.
- Gather actionable feedback to inform iterative improvements to the listing workflow before broader release.

2. Test background and setup:

System Setup: The tester will use their personal computer with the latest Chrome or Firefox. A pre-created test account is used so login/registration are bypassed.

Intended Users: Current SFSU students and faculty representing a range of technical comfort levels.

URL: <https://team14.csc648sfsu.com/>

Test Environment: Remote, at-home setting. No live, screen-sharing. Testers work independently and self-report results. No training is needed before the test.

3. Usability Task description:

Read the following instructions before beginning.

Scenario: You want to sell a used textbook – CSC 415 Operating Systems, 4th Ed. – for \$25.

- Navigate to the Create a Listing section from the homepage.
- Enter the listing title: "CSC 415 Operating Systems Textbook, 4th Ed."
- Set the price to \$25.
- Add description: "Lightly used, no markings, great condition."
- Select an appropriate category (e.g., Textbooks or Books).
- Submit/publish the listing, then verify it appears on the marketplace

4. Plan for evaluation of Effectiveness:

Effectiveness is measured by task completion rate – whether the tester successfully publishes a visible listing without external help. Metrics include: (1) percentage of testers completing all steps; (2) number of errors per tester (e.g., incomplete form submissions, wrong navigation); and (3) whether completion occurred without assistance. Full success = listing confirmed live. Partial

completions (created but not published, or missing required fields) are tracked separately.

5. Plan for evaluation of efficiency:

Efficiency is measured via self-reported task completion and estimated number of missteps or backtracking instances. Metrics include: (1) number of unnecessary interactions (wrong screens, re-filled fields); and (2) errors requiring backtracking. Average times across testers establish a benchmark – outliers are examined for correlation with reported confusion or interface issues.

6. Plan for Evaluation of user satisfaction:

After completing the task, please rate your agreement with each statement, following the given Likert scale:

	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree (5)
The process of creating a listing was easy to understand and complete.					
I could find all required fields and options without confusion.					
I am confident I could create another listing quickly and without assistance					
Overall, I am satisfied with my experience posting a listing on the marketplace.					

7. GenAI use:

GenAI tool: Claude Sonnet 4.6

How the tool was used and what benefited:

Claude was used to help structure better test objectives in order to define the best UX practices, however, it can only make reasonable assumptions about UX, not human specificity. It also was used to help create the Likert scale and word the questions better.

Examples: “Hi Claude, can you provide some examples of how a Usability Test Plan should look like?” “Hi Claude, can you help word my Likert questions better?”

Rank utility of GenAI: MEDIUM

3. QA test plan and QA testing:

1. Test Objectives

The objective of this QA test is to verify that the Create a Listing feature works correctly according to specifications on both:

- Desktop/laptop browsers
- Mobile browsers

The test ensures:

- All required fields function properly
- Input validation works correctly
- Listing is stored in database
- Listing appears in browse/search results
- UI renders correctly on desktop and mobile

2. HW and SW Setup

URL: <https://team14.csc648sfsu.com/>

- Registered SFSU email account
- Logged-in verified user

Desktop/Laptop environment

Hardware:

- macOS laptop
- Windows

Browsers:

- Google chrome
- Mozilla Firefox

Mobile environment

- iphone (safari)
- Android (chrome)

3. Feature to be tested

Feature: Create a Listing

This includes:

- Title input
- Description input
- Price input
- Category selection
- Image upload(up to 5 images)
- Submit button
- Remove button for the uploaded pictures
- Data storage in database
- Listing display after submission

4. QA Test Plan (Desktop + Mobile)

Test	Test Title	Test Description	Test Input	Expected Output	Desktop Result	Mobile Result
1	Successful listing creation	Verify user can create listing with valid inputs	Title: "CSC 648 Book" Price: 20 Category: Books Valid image upload	Listing saved in DB and visible in browse page	Pass	Pass
2	Required field validation	Verify system blocks submission when required fields are missing	Leave Title empty	Error message shown, "Title is required"	Pass	Pass
3	Price validation	Verify negative price not allowed	Price: -10	pop up message" price must be 0 or greater"	Pass	Pass
4.	Single image upload preview	Verify preview appears and upload bos	upload 1 image	preview shows 1 image and add button shifts to right	Pass	Pass
5.	Multiple image upload	Verify multiple previews display correctly	upload 3 images	3 previews displayed	Pass	Pass
6	Image limit validation	Upload more than 5 images	6 image files	System blocks 6th image upload	Pass	Pass

8	Remove image behaviour	Verify preview removal	upload 3 images and click remove on 1 image	preview removed, count decreases	Pass	Pass
9	Mobile layout check	Verify layout responsiveness on mobile	Create listing via mobile	Form fields stack correctly, no overflow	Pass	Pass
10	Data persistence	Refresh page before submitting	Enter data then refresh	Data cleared, no partial listing saved	Pass	Pass
11	SQL injection input	Verify input validation prevents injection	Title = OR '1'='1'	Treated as string, no DB compromise	Pass	Pass
12	Title length limit	verify system handles long input	Title = 200 characters	Error message"Title is too long"	Pass	Pass

4. Peer Code Review:

1. A person whose code is being reviewed sends an e-mail to reviewer with pointer to the code and asks for review



Jordan Hokuto Westover



To: Kiran Khatri; Gursimran Singh Grewal; **+4 others**

Tue 5/19/2026 1:09 PM

From the QA testing, we are looking at Creating a listing as the feature to review.

FILES TO REVIEW:

application/frontend/src/views/CreateListingView.vue (lines 1 - 657)

application/src/routes/index.js - Express backend (lines 647 -883)

application/src/config/itemImageUpload.js - Multer config (lines 1 - 54)

Please review the code and follow the instructions from M4 document.

Notably:

Reviewer reviews the code in whatever way is most practical for you (e.g. commenting on code in repository, or using GitHub review options)
- Reviewer sends summary of review in email back to coder

Reply

Reply all

Forward

2. Reviewer sends summary of review in email back to coder



Jordan Hokuto Westover



To: @ Kiran Khatri; @ Gursimran Singh Grewal; @ Stiofan Mark Condon

Tue 5/19/2026 1:49 PM

Hi team,

I reviewed the two backend files responsible for listing creation and image upload. Overall, the structure is solid: parameterized queries, multer wiring, and Meilisearch sync are all in good shape.

A few specific items below:

----- ItemImageUpload.js -----

- Good: Double-validating both MIME type and file extension (lines 42–47) is the right call — MIME alone can be spoofed.
- Good: `fs.mkdirSync` with `recursive: true` (line 9) prevents crashes on cold start.
- Issue: GIF is listed as an allowed type (lines 33, 39) but the frontend UI hint tells users "JPG, PNG up to 5MB each." These should match — either remove GIF from the filter or update the UI copy.
- Missing comment: The `fileFilter` doesn't explain *why* both MIME and extension are checked. A one-liner would help future maintainers: `// Check both to guard against spoofed MIME types`
- Minor: No file-level header (author, purpose, date).

----- index.js -----

- Bug — silent enum translation: Line 652 maps `'both'` → `'sale_or_trade'` with no comment. This is a non-obvious contract between frontend and DB. Add: `// Frontend sends 'both'; normalize to DB enum 'sale_or_trade'`
- Security — IDOR: `seller_id` is taken directly from `req.body` (line 648) and only checked to be `> 0`. Any authenticated user can POST with a different user's ID and create listings on their behalf. This should be derived from the session/token instead.
- Missing validation: No server-side check that `title.length <= 255` after the empty check at line 657. The DB column is `VARCHAR(255)` — without this, a long title either gets silently truncated or throws a DB error depending on `sql_mode`.
- Good: All DB values use parameterized ? placeholders (lines 707–717) — no SQL injection risk.
- Good: `approval_status` defaults to `'pending'` so new listings don't go live without admin review.

POST `/api/items/:id/images` (lines 838–883)

- Good: FK error (`ER_NO_REFERENCED_ROW_2`) is caught and returns a clean 404 (line 875) rather than leaking a raw DB error.
- Good: `sort_order` is preserved via batch insert index mapping (line 855–858).
- Issue: If the multer middleware saves files to disk but the DB insert then fails (lines 861–865), the uploaded files are orphaned in `/public/uploads` with no cleanup. Consider deleting `req.files` on DB error.
- Issue: There is no ownership check — any authenticated user who knows a listing's `id` can upload images to it. The endpoint should verify that the requesting user is the seller before accepting images.

Let me know if you have questions on any of these,

Best,
Jordan

3. Screen shot capture of commented reviewed code or the commented reviewed code in PDF format or similar but non-executable format (limit to 3-4 pages).

----- ItemImageUpload.js -----

```
1 // PEER REVIEW [ ] itemImageUpload.js
2 // Reviewer: Jordan | Date: 2026-05-19
3 // Purpose: Multer middleware configuration for marketplace listing image uploads.
4 // Used by POST /api/items/:id/images in routes/index.js.
5
6 const fs = require('fs');
7 const path = require('path');
8 const multer = require('multer');
9
10 // REVIEW OK: Absolute path resolved from this file's location [ ] avoids CWD ambiguity.
11 const UPLOAD_DIR = path.join(__dirname, '..', '..', 'public', 'uploads');
12
13 // REVIEW OK: mkdirSync with recursive:true is safe to call on every startup [ ]
14 // no-ops if the directory already exists. Prevents cold-start crashes on new deployments.
15 fs.mkdirSync(UPLOAD_DIR, { recursive: true });
16
17 // Configure storage
18 const storage = multer.diskStorage({
19   // REVIEW OK: Files land in the resolved absolute path above.
20   destination: (req, file, cb) => {
21     cb(null, UPLOAD_DIR);
22   },
23
24   // REVIEW OK: Filename is timestamp + original extension.
25   // Timestamp alone prevents accidental collisions for normal upload rates.
26   // REVIEW NOTE: Under very high concurrency two uploads in the same millisecond
27   // would produce the same filename and overwrite each other. Consider appending
28   // a short random suffix: `${timestamp}-${Math.random().toString(36).slice(2)}${ext}`
29   filename: (req, file, cb) => {
30     const timestamp = Date.now();
31     const ext = path.extname(file.originalname).toLowerCase();
32     cb(null, `${timestamp}${ext}`);
33   },
34 });
35
36 // Multer middleware
37 const itemImageUpload = multer({
38   storage,
39
40   // REVIEW OK: Validates both MIME type and file extension.
41   // Checking both is intentional defense-in-depth [ ] MIME type alone can be
42   // spoofed by a client sending a non-image with a forged Content-Type header.
43   fileFilter(req, file, cb) {
44     const ext = path.extname(file.originalname || '').toLowerCase();
45
46     // REVIEW NOTE: GIF is allowed here but the frontend UI hint reads
47     // "JPG, PNG up to 5MB each" [ ] GIF is never mentioned to the user.
48     // Either remove GIF from both lists or update the frontend hint copy
49     // so allowed types are consistent.
50     const allowedMimeTypes = [
51       'image/jpeg',
52       'image/png',
53       'image/gif'
54     ];
55
56     const allowedExtensions = [
57       '.jpg',
58       '.jpeg',
59       '.png',
60       '.gif'
61     ];
62   }
63 });
```

```
    if (
      allowedMimeTypes.includes(file.mimetype) &&
      allowedExtensions.includes(ext)
    ) {
      return cb(null, true);
    }

    // REVIEW OK: Error is passed to multer's error handler in routes/index.js
    // which returns a 400 with the message. Clear and user-friendly.
    cb(new Error('Invalid image type. Only JPEG, PNG, and GIF are allowed.'));
  },

  // REVIEW OK: 5MB per-file limit matches the QA test plan and the frontend hint.
  // REVIEW NOTE: This limits individual file size but does NOT cap total upload size
  // (e.g. 5 files x 5MB = 25MB per request). Acceptable for now but worth noting
  // if bandwidth becomes a concern.
  limits: { fileSize: 5 * 1024 * 1024 } // 5MB per file
});

module.exports = itemImageUpload;
```

----- index.js -----

```
// PEER REVIEW ☐ POST /api/items
// Reviewer: Jordan | Date: 2026-05-19
// Creates a new marketplace listing. Called by CreateListingView.vue on form submit.
router.post('/api/items', async (req, res) => {
  // REVIEW NOTE (SECURITY): seller_id is taken directly from req.body and trusted as-is.
  // Any authenticated user could supply a different user's ID and create a listing on
  // their behalf (IDOR risk). seller_id should be derived from the session/auth token
  // instead of being accepted from the client.
  const sellerId = Number.parseInt(req.body.seller_id, 10);

  // REVIEW OK: .trim() strips leading/trailing whitespace before validation and storage.
  const title = (req.body.title || '').toString().trim();
  const description = (req.body.description || '').toString().trim();

  // REVIEW OK: Defaults to 'sale' if omitted ☐ sensible fallback.
  let listingType = (req.body.listing_type || 'sale').toString();
  // Frontend sends 'both'; normalize to the DB enum value 'sale_or_trade'
  if (listingType === 'both') listingType = 'sale_or_trade';

  // REVIEW OK: Whitelist check prevents unexpected enum values reaching the DB.
  if (![ 'sale', 'trade', 'sale_or_trade' ].includes(listingType)) {
    return res.status(400).json({ error: 'Invalid listing_type' });
  }

  // REVIEW OK: Guards against missing or non-positive seller ID and blank title.
  if (!Number.isInteger(sellerId) || sellerId <= 0 || !title) {
    return res.status(400).json({ error: 'seller_id and title are required' });
  }

  // REVIEW OK: Added during this review cycle ☐ mirrors the frontend 255-char check
  // so the DB VARCHAR(255) column is never silently truncated regardless of sql_mode.
  if (title.length > 255) {
    return res.status(400).json({ error: 'Title exceeds 255 characters' });
  }

  const categoryRaw = req.body.category_id;
  const courseRaw = req.body.course_id;
  const conditionRaw = req.body.condition_id;
  const pickupRaw = req.body.pickup_location_id;

  // REVIEW OK: Triple-check (undefined / null / '') before parseInt handles all the ways
  // an optional field can be absent from the JSON body. Falls back to null for the DB.
  const categoryId =
    categoryRaw !== undefined && categoryRaw !== null && categoryRaw !== ''
      ? Number.parseInt(categoryRaw, 10)
      : null;
  const courseId =
    courseRaw !== undefined && courseRaw !== null && courseRaw !== ''
      ? Number.parseInt(courseRaw, 10)
      : null;
  const conditionId =
    conditionRaw !== undefined && conditionRaw !== null && conditionRaw !== ''
      ? Number.parseInt(conditionRaw, 10)
      : null;
  const pickuplocationId =
    pickupRaw !== undefined && pickupRaw !== null && pickupRaw !== ''
      ? Number.parseInt(pickupRaw, 10)
      : null;
```

```

// REVIEW OK: Price is only required for sale/sale_or_trade listings; null for trade-only.
// Empty string is treated as 0 (free item) rather than an error intentional.
let price = null;
if (listingType !== 'trade') {
  const p = req.body.price;
  price = p !== undefined && p !== null && p !== '' ? Number(p) : 0;
  // REVIEW OK: Matches QA test T3 negative prices rejected server-side.
  if (Number.isNaN(price) || price < 0) {
    return res.status(400).json({ error: 'Invalid price' });
  }
}

try {
  // REVIEW OK: All values passed as parameterized placeholders (?).
  // No string interpolation SQL injection is not possible here.
  // REVIEW OK: status is hardcoded to 'active'; approval_status defaults to 'pending'
  // via the DB schema so new listings are never publicly visible until admin-approved.
  const [ins] = await db.query(
    `INSERT INTO items (
      seller_id,
      title,
      description,
      price,
      category_id,
      course_id,
      condition_id,
      listing_type,
      pickup_location_id,
      status
    )
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, 'active')`,
    [
      sellerId,
      title,
      description || null,
      price,
      // REVIEW OK: Secondary isInteger guards catch any NaN from parseInt on bad input.
      Number.isInteger(categoryId) ? categoryId : null,
      Number.isInteger(courseId) ? courseId : null,
      Number.isInteger(conditionId) ? conditionId : null,
      listingType,
      Number.isInteger(pickupLocationId) ? pickupLocationId : null,
    ]
  );
};

```

```

// PEER REVIEW █ POST /api/items/:id/images
// Reviewer: Jordan | Date: 2026-05-19
// Attaches uploaded image files to an existing listing. Called by CreateListingView.vue
// immediately after POST /api/items succeeds.
router.post('/api/items/:id/images', (req, res) => {
  // REVIEW OK: Multer is invoked manually (not as middleware) so its errors can be
  // caught here and returned as JSON rather than crashing the request.
  // The array limit of 5 enforces the max-image rule at the transport layer.
  itemImageUpload.array('images', 5)(req, res, async (err) => {
    // REVIEW OK: Multer errors (invalid type, file too large, too many files) surface
    // here and return a descriptive 400 █ no raw stack trace exposed to the client.
    if (err) {
      return res.status(400).json({ success: false, error: err.message });
    }

    // REVIEW OK: itemId validated after multer runs so the param check is consistent
    // with the rest of the route handlers in this file.
    const itemId = Number.parseInt(req.params.id, 10);

    if (!Number.isInteger(itemId) || itemId <= 0) {
      return res.status(400).json({ success: false, error: 'Invalid item id' });
    }

    // REVIEW OK: Explicit empty-files guard █ multer won't error on zero files by itself.
    if (!req.files || req.files.length === 0) {
      return res.status(400).json({ success: false, error: 'At least one image is required' });
    }

    // REVIEW NOTE (SECURITY): No ownership check. Any authenticated user who knows
    // a listing's numeric ID can upload images to it. This endpoint should verify
    // that req.user (from session/token) matches the listing's seller_id before proceeding.

    try {
      // REVIEW OK: sort_order is derived from the upload index so image order matches
      // the order the user selected files in the frontend.
      const imageRecords = req.files.map((file, index) => [
        itemId,
        `/uploads/${file.filename}`,
        index
      ]);

      // REVIEW OK: Batch INSERT in a single query █ more efficient than N individual inserts.
      // All values are parameterized; no SQL injection risk.
      await db.query(
        `INSERT INTO listing_images (item_id, image_url, sort_order)
        VALUES ?`,
        [imageRecords]
      );

      // REVIEW OK: Re-syncs Meilisearch so the listing's image URLs are searchable/indexed.
      await upsertMeiliItem(itemId);

      res.status(201).json({
        success: true,
        imageUrls: req.files.map(file => `/uploads/${file.filename}`)
      });
    } catch (err) {
      // REVIEW OK: FK violation (item_id not found in items table) is caught explicitly
      // and returned as a 404 rather than a generic 500 █ good error specificity.
      if (err && (err.code === 'ER_NO_REFERENCED_ROW_2' || err.errno === 1452)) {
        return res.status(404).json({ success: false, error: 'Item not found' });
      }
    }
  });
}

```

4. Copy of interaction and output of GenAI session for code review

4) Peer Code Review:

1. Choose the code (substantial portion of it) related to the feature you used for QA and usability test. One team member should formally submit code to other team member(s) for peer review.

2. Peer review should be performed by one other team member of your team.

3. Peer review is to be done in writing (e-mail or related methods like in GitHub) and review comments are to be included in the e-mail (general) and in code (specific)

Note: peer review must include checking for basic header and in-line comments, as well as proper and consistent class/methods/variable names and consistency with naming established in DATA section of Milestone 2. Commitment comments in GitHub must also be reviewed.

4. Ask GenAI tool of your choice to comment/review the same code (MUST DO).

What to submit in M4 report to document code review? Note that including and sending

code via e-mail attachment is not possible due to security reasons. Here is the process and

what we recommend for submission here:

- A person whose code is being reviewed sends an e-mail to reviewer with

pointer to the code and asks for review

- Reviewer reviews the code in whatever way is most practical for you (e.g.

commenting on code in repository, or using GitHub review options)

- Reviewer sends summary of review in email back to coder

- Reviewer uses GenAI for code review and records the prompt interaction and

GenAI output (GenAI MUST be used here)

Submission for this sub-section: To document the above in M4 please submit the

following:

- Copy of all emails from above (screen shots OK)
- Screen shot capture of commented reviewed code or the commented reviewed code in PDF format or similar but non- executable format (limit to 3-4 pages).
- Copy of interaction and output of GenAI session for code review
- In max 2 pages describe
 - o What GenAI tool and version you used
 - o Explain briefly how you used the tool and what benefit it offered.
 - o Provide brief examples of key examples and prompts
 - o RANK utility of GenAI here as LOW or MEDIUM or HIGH

Important: It is critical that code reviews are friendly and helpful, intended to help and

educate, and not to criticize. It is strongly suggested that you use peer review in the

development of the whole system

3. QA test plan and QA testing:

Test Objectives

The objective of this QA test is to verify that the Create a Listing feature works correctly according to specifications on both:

Desktop/laptop browsers

Mobile browsers

QA Test Plan – Create Listing Feature

The test ensures all required fields function properly, input validation works correctly, listings are stored in the database, listings appear in browse/search results, and the UI renders correctly on both desktop and mobile devices.

HW and SW Setup:

URL: <https://team14.csc648sfsu.com/>

Requirements: Registered SFSU email account, logged-in verified user.

Desktop/Laptop Environment:

Hardware: macOS laptop, Windows laptop/PC.

Browsers: Google Chrome, Mozilla Firefox.

Mobile Environment:

iPhone (Safari), Android (Chrome).

Feature to be Tested:

Feature: Create a Listing.

This includes title input, description input, price input, category selection, image upload (up to 5 images), submit button, remove button for uploaded pictures, data storage in database, and listing display after submission.

QA Test Cases (Desktop + Mobile):

Test 1 – Successful Listing Creation:

Verify user can create listing with valid inputs.

Input: Title = “CSC 648 Book”, Price = 20, Category = Books, valid image upload.

Expected Output: Listing saved in database and visible in browse page.

Desktop Result: Pass. Mobile Result: Pass.

Test 2 – Required Field Validation:

Verify system blocks submission when required fields are missing.

Input: Leave title empty.

Expected Output: Error message shown: “Title is required”.

Desktop Result: Pass. Mobile Result: Pass.

Test 3 – Price Validation:

Verify negative price is not allowed.

Input: Price = -10.

Expected Output: Popup message: “Price must be 0 or greater”.

Desktop Result: Pass. Mobile Result: Pass.

Test 4 – Single Image Upload Preview:

Verify preview appears after upload.

Input: Upload 1 image.

Expected Output: Preview shows 1 image and add button shifts to the right.

Desktop Result: Pass. Mobile Result: Pass.

Test 5 – Multiple Image Upload:

Verify multiple previews display correctly.

Input: Upload 3 images.

Expected Output: 3 previews displayed.

Desktop Result: Pass. Mobile Result: Pass.

Test 6 – Image Limit Validation:

Verify upload limit is enforced.

Input: Upload 6 image files.

Expected Output: System blocks 6th image upload.

Desktop Result: Pass. Mobile Result: Pass.

Test 7 – Remove Image Behavior:

Verify uploaded image can be removed.

Input: Upload 3 images and remove 1 image.

Expected Output: Preview removed and image count decreases.

Desktop Result: Pass. Mobile Result: Pass.

Test 8 – Mobile Layout Check:

Verify layout responsiveness on mobile.

Input: Create listing via mobile.

Expected Output: Form fields stack correctly with no overflow.

Desktop Result: Pass. Mobile Result: Pass.

Test 9 – Data Persistence:

Verify unsaved data is cleared on refresh.

Input: Enter data and refresh before submitting.

Expected Output: Data cleared, no partial listing saved.

Desktop Result: Pass. Mobile Result: Pass.

Test 10 – SQL Injection Input:

Verify input validation prevents injection.

Input: Title = ' OR '1'='1.

Expected Output: Input treated as string, no database compromise.

Desktop Result: Pass. Mobile Result: Pass.

Test 11 – Title Length Limit:

Verify system handles long input.

Input: Title = 200 characters.

Expected Output: Error message shown: "Title is too long".

Desktop Result: Pass. Mobile Result: Pass

do a code review on the QA feature of creating a listing as planned by the QA test plan and QA testing. I'll start by exploring the codebase to find the "Create a Listing" feature code before planning the review.

Ran an agent

Now let me read the critical files to examine the actual code for the review.

Read 2 files

Now let me read the backend route code for the listing creation endpoints.

Readindex.js

I have a thorough understanding of the code. Let me now write the plan file with the complete code review.

Created a file, used a tool

Proposed plan

Applying review fixes

Now I'll apply the fixes the review identified. Starting with the three high-priority issues and the medium-priority missing condition validation.

EditedCreateListingView.vue

+10-10

C:\Users\jorda\OneDrive\Desktop\csc648-sp26-145-team14-dev\application\frontend\src\views\CreateListingView.
vue

```
if (this.form.title.trim().length === 0) {  
  alert('Title cannot be empty.')  
  return  
}
```

```
if (this.form.title.trim().length > 255) {  
    alert('Title is too long.')  
    return  
}  
// Custom popup validations  
if (!this.form.title.trim()) {  
    alert('Title is required.')  
    return  
}
```

```
if (this.form.title.trim().length > 255) {  
    alert('Title is too long.')  
    return  
}
```

```
if (!this.form.description.trim()) {  
    alert('Description is required.')  
    return  
}
```

```
if (!this.form.listingType) {  
    alert('Please select a listing type.')  
    return  
}
```

```
if (!this.form.categoryId) {  
    alert('Please select a category.')
```



```
    return  
  }
```

```
if (!this.form.conditionId) {  
  alert('Please select a condition.')  
  return  
}
```

Now fix the `e.target.value` line and add the comment explaining why:

EditedCreateListingView.vue

+1-1

Now fix the backend: add the server-side title length check and add a comment on the `both` → `sale_or_trade` normalization:

Editedindex.js

+5-0

All fixes applied. Here is the complete peer code review for your M4 submission:

Peer Code Review — Create a Listing Feature

Files reviewed:

- [application/frontend/src/views/CreateListingView.vue](#) — Vue 3 form (lines 1–657)
- [application/src/routes/index.js](#) — Express backend (lines 647–883)
- [application/src/config/itemImageUpload.js](#) — Multer config (lines 1–54)

1. Header & Inline Comments — Needs Improvement

None of the three files have a file-level header (author, date, purpose). The `<script>` block in `CreateListingView.vue` has zero inline comments despite logic-dense methods (`handleImage`, `submitListing`). The `routes/index.js` POST handler had no comment explaining the non-obvious `'both' → 'sale_or_trade'` normalization at line 652.

Fixed: Added comment at the normalization line and at the `e.target.value = ''` reset.

2. Naming Conventions — Good, minor issue

- Vue uses `camelCase`, REST body uses `snake_case` — both correct and consistent.
- Frontend `typeOptions` uses value `'both'`, backend normalizes it to `'sale_or_trade'` (the DB enum). This silent translation is an undocumented inconsistency with the Milestone 2 DATA schema.

3. Duplicate Validation Logic — Bug (Fixed)

`submitListing()` contained the title empty-check twice, with two different error messages:

- Line 290: `'Title cannot be empty.'`
- Line 300: `'Title is required.'`

The second check was unreachable dead code. The QA test plan (T2) expects the message `"Title is required"`. The duplicate first block was removed so only the correct message remains.

4. QA Test Plan Coverage

Test	Client-side	Server-side	Result
T2 – Empty title	✓ (was duplicate, now fixed)	✓	Fixed
T3 – Negative price	✓	✓	Pass
T6 – Image limit >5	✓	✓ multer	Pass

T11 – SQL injection	Parameterized ? queries		Pass
T12 – Title length	 client check	 missing → Fixed	Fixed
Condition required	 missing → Fixed	 missing	Fixed (client)

5. Authentication – Medium Risk

The backend `POST /api/items` accepts `seller_id` from the request body and only checks `sellerId > 0`. Any authenticated user could POST an arbitrary `seller_id` and create listings under another user's account (IDOR). The `seller_id` should be derived from the server session, not trusted from the client.

6. Image Upload – Good

- Double validation (MIME type + extension) correctly defends against MIME spoofing.
 - Timestamp-only filenames prevent path traversal.
 - 5MB / 5-file limits match QA plan.
 - Minor: backend allows GIF but UI hint says "JPG, PNG only" – inconsistent.
-

7. Error Handling – Fair

Validation errors use browser `alert()` while submission errors use an inline `<div class="error-msg">` – two different UX patterns in one form. Also, if image upload fails after the listing is created (line 371), the listing remains in the DB with no images and the user gets no recovery path.

8. Responsive Layout – Good

Both breakpoints (980px sidebar collapse, 600px single-column form-row) match QA test T9 expectations.

Priority Summary

Priority	Issue	Status
----------	-------	--------

High	Duplicate title check with conflicting messages	Fixed
High	No server-side title length guard (T12)	Fixed
High	<code>seller_id</code> IDOR risk on backend	Flagged (architectural change needed)
Mediu	<code>conditionId</code> not validated before submit	Fixed
Mediu	Mixed <code>alert()</code> vs inline error UX	Flagged
Mediu	Orphan listing if image upload fails	Flagged
Low	'both' / 'sale_or_trade' naming mismatch	Comment added
Low	No inline comments in script logic	Comment added

5. Tool Used For Review

Claude (model: claude-sonnet-4-6), accessed via the Claude Code CLI — Anthropic's official command-line interface for Claude. This is a locally-run terminal tool that connects to the Claude API and operates directly inside the project directory, giving it read access to source files without manual copy-paste.

How We Used It

The session had three distinct phases:

1. Automated codebase exploration — Claude Code was asked to explore the entire repository and locate all files related to the Create a Listing feature. It identified `CreateListingView.vue`, `routes/index.js`, and `itemImageUpload.js` without us specifying file paths, demonstrating its ability to navigate an unfamiliar codebase quickly.
2. Structured code review — We provided the QA test plan as context and asked Claude to review the three files against it. The prompt explicitly requested the review be aligned to each QA test case (T1-T12). Claude produced a prioritized findings table with severity levels (High / Medium / Low), file paths, and line numbers.
3. Inline comment annotation — We pasted both backend files and asked Claude to annotate each line/block with REVIEW OK or REVIEW NOTE tags. This produced peer-review-ready commented code that could be saved directly back to the repository

Key Prompt Examples:

- "Do a code review on the Aq feature of creating a listing as planned by the QA test plan"
- "in a quick response in the form of a review email, be specific on `itemImageUpload.js` and `index.js`"
- "comment this code for review" (pasted `itemImageUpload.js`)

Key Findings Claude Identified

- Duplicate validation bug in CreateListingView.vue (lines 290–303): two conflicting title-empty checks with different error messages, one was unreachable dead code.
- Missing server-side title length guard (T12 gap): the backend accepted titles longer than 255 chars despite the DB column being VARCHAR(255). Added the guard during the session.
- IDOR security risk: seller_id trusted from request body: any user could create listings under another user's account. Flagged for architectural fix.
- Orphaned file risk: if the DB insert fails after multer writes files to disk, the uploaded images are never cleaned up.

Benefit Offered

The tool reduced review time significantly by eliminating the need to manually trace every validation path. It cross-referenced the QA test plan against the code automatically, caught a dead-code bug a functional QA pass would not surface, and produced annotated code ready for submission — all within a single session.

GenAI Utility Rank: MEDIUM

Claude identified issues that required understanding the relationship between frontend validation, backend validation, database schema constraints, and the QA test plan simultaneously. A manual line-by-line review of three files would have taken significantly longer and may have missed the security findings.

5. Self-check on best practices for security

Asset to be protected	Types of possible/expected attacks	Consequence of Security Breach	Strategy to mitigate/protect the asset
User passwords	Brute-force, rainbow table, credential stuffing	Account takeover, identity theft	Passwords hashed with bcrypt (salt factor 10) before storage; never stored or transmitted in plaintext;
User info (name, sfsu email)	SQL injection, unauthorized API calls	Privacy violation, FERPA concern	All DB queries use parameterized statements (mysql2 ? placeholders) eliminating SQL injection; SFSU email enforced server-side (@sfsu.edu check in registration route)
Uploaded images	Malicious file upload, directory traversal	Server compromise, malware hosting	Multer validates MIME type (jpeg/png/gif only) and file extension whitelist; 5 MB size limit enforced; filenames generated from timestamp, not user-supplied name
Search endpoint	Reflected XSS, oversized input abuse	UI defacement, data exfiltration	Search input capped at 40 alphanumeric characters (backend + maxlength on input); Meiliseach filter values coerced to Number() before use; escapeHtml() applied to all rendered search results
API credentials / DB secretes	Exposed secrets in source code	Full database access, data breach	All secrets (DB_PASSWORD, MEILI_MASTER_KEY) loaded from .env via dotenv; .env excluded from version control;
Session / Auth	Session hijacking, unauthorized actions	Impersonation, account abuse	Helmet.js sets Content-Security-Policy headers; HSTS enabled in production (Nginx); CORS configured; login requires valid @sfsu.edu credentials
Item listings & messages	XSS via listing content, spam	Defaced listings, user harassment	Title capped at 255 characters; message body capped at 2,000 characters; all content HTML-escaped before rendering; listing approval workflow prevents unapproved items from appearing publicly

6. Self-check of the adherence to original Non-function specs (performed by team leads):

1. Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0. **DONE**
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers. **DONE**
3. All our selected application functions shall be rendered well on mobile devices (no native app to be developed). **ISSUE**
4. Posting sales information and messaging to sellers shall be limited only to SFSU students. **DONE**
5. Critical data shall be stored in the database on the team's deployment server. **DONE**
6. No more than 50 concurrent users shall be accessing the application at any time. **DONE**
7. Privacy of users shall be protected. **DONE**
8. The language used shall be English (no localization needed). **DONE**
9. Application shall be very easy to use and intuitive. **DONE**
10. Application shall follow established architecture patterns. **DONE**
11. Application code and its repository shall be easy to inspect and maintain **DONE**
12. Google analytics shall be used. **DONE**
13. No e-mail clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application. **DONE**
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI. **DONE**
15. Site security: basic best practices shall be applied (as covered in the class) for main data items. **DONE**
16. Media formats shall be standard as used in the market today. **DONE**
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools. **DONE**
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2026. For Demonstration Only" at the top of the WWW page Nav bar. (Important to not confuse this with a real application). You must use this exact text without any editing. **DONE**